

Project Report: AI Architecture Agent

Team: - **Anjaneyulu** (Backend, Database, Authentication & History Sidebar) - **Kotha Ashwin** (AI Core, Multi-Agent Orchestrator & React Flow Canvas)

Project Overview

The **AI Architecture Agent** is an autonomous, self-healing, cloud system design workspace. It serves as a visual and cognitive playground where developers can upload system requirements (PDFs/Docs), trigger a pipeline of collaborative AI agents, and instantly generate interactive architectural canvases, REST API routers, database schemas, cost projections, and deployable Terraform Infrastructure as Code (IaC) blueprints.

Technology Stack

- **Backend Server & Gateway:** Python, FastAPI, Uvicorn
- **Database Layer:** SQLite, SQLAlchemy ORM
- **AI Orchestration & Agents:** Llama-3.3-70b (via Groq API), Google Gemini API
- **Frontend Dashboard:** React 19, Vite, Zustand (State Management)
- **Visual Canvas Engine:** React Flow
- **Real-Time Data Transport:** HTTP Streaming & SSE (Server-Sent Events)
- **Security & Session Layer:** JWT (JSON Web Tokens), Bcrypt encryption

Project Scope

In-Scope

- **FastAPI Core Service:** Hosting user accounts, project workspaces, document storage, and chat history.
- **Requirements Parser & Embeddings:** Parsing uploaded files and caching text metadata.
- **Visual Node Canvas:** Rendering system topologies, database relational models, and API endpoints with interactive edges.
- **Collaborative Agent Orchestration:** Coordinating 10+ specialized agents (Security, Cost, Database, API, etc.) sequentially.
- **Interactive History Sidebar:** Restoring documents and conversations across refreshes.

Out-of-Scope

- Running `terraform apply` or provision real resources automatically.
- Route actual client traffic to generated services.
- Mobile layout optimization for the canvas board.

Domain 1: FastAPI API Gateway (Backend)

This tier serves as the application's central nervous system, managing accounts, databases, and routing state changes. - **API Gateway & Routers:** FastAPI backend managing routes for `/auth` (login/register), `/projects` (CRUD workspaces), `/chat`

(interactive queries), and `/analysis` (pipeline states). - **Relational SQLite Store (SQLAlchemy)**: Persists user accounts (`User`), project workspaces (`Project`), uploaded documents (`Document`), chat threads (`Conversation`), message logs (`Message`), and generated topologies (`ArchitectureVersion`). - **Auth Guard Middleware**: A JWT middleware interceptor decrypting authentication headers and blocking unauthorized requests.

Domain 2: Autonomous AI Core (Intelligence)

This tier houses the multi-agent orchestrator that designs the system architecture. - **Agent Orchestrator Loop**: Coordinates agents in a sequential flow, passing input contexts (functional specs) from one agent to the next (e.g., Requirement → Database → API → Cloud → Security → Cost → Terraform → Review). - **RAG & Guidelines Search**: Interfaces with vector embeddings to match designs against pre-defined compliance standards. - **Rotational API Key Pooling**: A key-rotation mechanism that switches Groq/Gemini API keys upon encountering rate limit errors (429), preventing compilation crashes.

Domain 3: Interactive React Flow Dashboard (Frontend)

This tier provides the visual workspace where users analyze, modify, and review system topologies. - **React Flow Diagram Canvas**: Renders interactive visual components, API routers, and database tables. Features handle alignment, horizontal API nodes, and animated edges. - **Interactive History Sidebar**: A left-panel navigation dashboard showing past conversations and uploaded documents. - **Chat Assistant Panel (AiPanel)**: A real-time chat interface displaying user-bot messages, streaming completions, and auto-saving conversations to the backend.

The Operational Data Flow Pipeline

Phase 1: Context Ingestion & Project Setup

- **Step 1 (Upload)**: User logs in and uploads a requirements document (PDF/Doc) to the workspace.
- **Step 2 (Parsing)**: Backend parses text from the file and saves the document record into SQLite.
- **Step 3 (Restoration)**: Frontend fetches the document list and displays it inside the left sidebar.

Phase 2: Agent Orchestration & Generation

- **Step 1 (Triggers)**: User clicks "Run Full Pipeline" on the dashboard.
- **Step 2 (Execution)**: The backend triggers the Agent Orchestrator. Agents execute in order to draft components, SQL schemas, endpoints, and Terraform.
- **Step 3 (Commit)**: The final reviewed version is saved as an `ArchitectureVersion` in SQLite.

Phase 3: Visual Canvas Rendering & Active Chat

- **Step 1 (Draw)**: Frontend fetches the output and renders it as components, database tables, and API nodes on the canvas.
 - **Step 2 (Chat)**: User inputs a modification request (e.g. *"add redis caching"*).
 - **Step 3 (Update)**: AI streams the response, updates the project state, and saves both messages to the database.
-

Project Plan & Team Contributions

Anjaneyulu (Backend, Database, Auth & History Sidebar)

- **Focus:** Created the initial Express backend, migrated the complete backend to FastAPI, designed database schemas, implemented JWT authentication, and built the interactive sidebar workspace history.
- **Skills Learned:** FastAPI routing, SQLAlchemy relations, SQLite database modeling, Zustand state synchronization, and JWT route protection.
- **Challenges Overcome:** Standardized decoded token payloads to solve `/auth/me` crashes, resolved Vite compilation failures (`MISSING_EXPORT` typo), and structured streamed AI chunks to commit to SQLite database logs seamlessly.

Kotha Ashwin (AI Core, Multi-Agent Orchestrator & React Flow Canvas)

- **Focus:** Developed the core multi-agent Python framework, configured Groq/Gemini LLM client interfaces, and built the visual canvas interface using React Flow.
- **Skills Learned:** Multi-agent prompt engineering, React Flow custom node and edge styling, and dynamic coordinate layouts.
- **Challenges Overcome:** Eliminated LLM hallucinated outputs by implementing strict parseable schemas in templates, and fixed node handle alignment for horizontal API endpoint nodes.

Integration Seams (How It Came Together)

1. **Phase 1 → Phase 2:** The backend exposes `GET /projects/{id}/documents` and `GET /projects/{id}/messages` endpoints. The frontend dev integrates these calls inside `useStore.js` and `AiPanel.jsx` to load and maintain user contexts on project switch.
2. **Phase 2 → Phase 3:** The Agent Orchestrator outputs structured JSON payloads containing components, database schemas, and APIs. The frontend canvas dev parses this JSON to render nodes and connect edges.
3. **The Shared Chat Stream:** The backend streams chat responses chunk-by-chunk using Event Streams (SSE). The frontend UI decodes this stream to draw real-time text, and once the stream completes, the backend automatically writes the entire chat log to the SQLite `Message` table.

AI	ARCHITECTURE	AGENT	ARCHITECTURE
----	--------------	-------	--------------

```

+-----+
| DOMAIN 3: INTERACTIVE REACT FLOW DASHBOARD |
|
| [React Vite UI Dashboard] <---> [Zustand State Store] <---> [React Flow Canvas] |
|
+-----+
|
| (HTTP Requests / SSE Chat Streams / JWT Auth)
|
+-----+
|
| v
| [Auth Middleware] <---> [API Gateway Routers] (FastAPI run.py) |
|
|
| DOMAIN 1: FastAPI API GATEWAY
|
| [SQLAlchemy SQLite] <=====> [SQLite Database (arch_agent.sqlite)] |
+-----+
|
| (Trigger Analysis Pipeline) | (Read/Write Version)
|
| v
+-----+
|
| v
| [Agent Orchestrator Loop]
|
| [RAG Embeddings (Pinecone)] <---> [LLM Client Engine (Groq/Gemini APIs)]
|
|
| DOMAIN 2: AUTONOMOUS AI CORE
|
+-----+

```